ENTREGA DE ATIVIDADE — GRUPO 05

COMPONENTE CURRICULAR:	Práticas Profissionais em Análise e Desenvolvimento de Sistemas
NOME COMPLETO DO ALUNO:	Bruno Moreira da Silva
RA:	10723746

Atenção: Toda atividade deverá ser feita com fonte Arial, tamanho 11, espaço de 1,5 entre as linhas e alinhamento justificado entre as margens.

Atividade: Aplicando Conhecimento — Especificação revisada da Iteração 2

ValidaHoras | Grupo 05 | São Paulo, 2026

Integrantes

João Victor Santos Pinto Sotonyi — RA 10720147

Bruno Moreira da Silva — RA 10723746

Rafael Silva de Barros — RA 10721409

Murillo Henrique Santos — RA 10720644

Guilherme Pedrotti Perossi — RA 10443090

Repositório de entrega: https://github.com/murilloohs/ppads-grupo-05

Data da revisão: 04 de novembro de 2026.

1. Visão geral, interessados e evolução

O ValidaHoras registra atividades complementares e seus comprovantes, compara as informações com regras institucionais e permite revisão humana das solicitações. O aluno acompanha protocolos, decisões e histórico. O objetivo é apoiar a conferência acadêmica e reduzir o trabalho repetitivo, mantendo a decisão justificável.

Interessados: alunos, funcionários avaliadores, administradores acadêmicos, coordenação de curso, equipe de tecnologia e instituição de ensino. O cadastro e a análise são isolados por aluno e instituição, conforme o perfil autenticado.

1.1 Histórico de revisão

Referência	Conteúdo
Iteração 1 / v1	Login, cadastro, upload, revisão antes do envio, protocolo e acompanhamento inicial.
Iteração 2 / v2	OCR e regras, decisão manual, detalhes e histórico, administração versionada, correção e reenvio, integridade e instalação documentada.

Os documentos anteriores de requisitos e modelagem foram preservados no repositório. Esta revisão consolida o comportamento da entrega atual e substitui, para esta versão, as descrições anteriores que indicavam OCR, revisão ou telas ainda como pendências.

1.2 Escopo da entrega

Inclui os cinco casos de uso principais, três perfis de demonstração, persistência PostgreSQL, interface React, API FastAPI, migrações e instruções de instalação. A aplicação é acadêmica e local. Integração com sistemas institucionais, relatórios gerenciais, gestão administrativa completa de cursos/usuários e implantação pública são evoluções futuras.

2. Requisitos funcionais e critérios de aceite

ID	Requisito
RF01	Autenticar usuários e aplicar autorização por perfil.
RF02	Cadastrar atividade com nome, categoria, instituição, período e horas.
RF03	Receber certificado PDF, JPG ou PNG, com limite padrão de 10 MB.
RF04	Validar conteúdo, tamanho, integridade e repetição exata do comprovante do aluno.
RF05	Gerar protocolo e registrar cada envio com vínculo ao aluno.
RF06	Extrair texto e campos do documento por leitura digital ou OCR.
RF07	Comparar dados declarados, dados extraídos e regra vigente aplicável.
RF08	Registrar resultado, explicação, comparações e confiança da análise.
RF09	Aprovar automaticamente quando todos os critérios coincidirem e a regra permitir; nos demais casos, encaminhar para revisão.
RF10	Permitir aprovação, reprovação ou correção pelo avaliador, com justificativa.
RF11	Consultar solicitações próprias, pesquisar, filtrar, baixar comprovante e acompanhar histórico.
RF12	Cadastrar regras e criar versões sem sobrescrever critérios históricos.
RF13	Preservar decisões, status, responsável e evidências de cada envio/reenvio.

Aceite principal: um aluno envia certificado real, recebe análise, pode consultar o resultado e, quando solicitado, corrigir e reenviar. Um avaliador de outra instituição não acessa a fila. Editar uma regra não altera a versão associada a uma solicitação anterior.

3. Regras de negócio e atributos de qualidade

• RB01 — Somente o aluno proprietário pode cadastrar, editar e consultar sua atividade. Edição e upload ficam fechados após envio, salvo solicitação de correção.

• RB02 — Arquivo vazio, inválido, corrompido, protegido ou acima do limite é rejeitado. Um comprovante com o mesmo SHA-256 em outra atividade do mesmo aluno é bloqueado; isso não detecta todas as formas de duplicidade semântica.

• RB03 — Cada envio associa documento e retrato dos dados da atividade à solicitação. Correções geram nova solicitação/protocolo na mesma atividade, e a consulta reúne o histórico.

• RB04 — A regra aplicável é selecionada pela instituição, categoria e vigência. Entre versões aplicáveis, utiliza-se a mais recente. Edição cria nova versão e registra o autor; não modifica avaliações anteriores.

• RB05 — Aprovação automática exige regra permissiva, confiança suficiente e nenhuma comparação divergente ou inconclusiva. Divergências, ausência de regra e falha de leitura seguem para revisão.

• RB06 — O avaliador decide somente pedidos pendentes da sua instituição. Justificativa é obrigatória; uma decisão concorrente ou posterior encontra conflito, em vez de sobrescrever a primeira.

3.1 Requisitos não funcionais

A interface utiliza linguagem em português, rótulos de formulário, navegação por teclado e estados de erro/carregamento. Autenticação JWT e autorização são verificadas na API. O sistema registra histórico e verifica a integridade no acesso ao certificado. A organização em serviços separa análise, OCR, revisão e regras.

A meta inicial de resposta em até três segundos não é certificada nesta entrega: OCR depende do arquivo e do ambiente. Segurança de produção, disponibilidade, capacidade, acessibilidade integral e conformidade com a LGPD exigem validações próprias antes de implantação institucional. As contas e certificados de demonstração não utilizam dados acadêmicos reais.

4. UC01 — Submeter atividade complementar

Ator: aluno autenticado e ativo. Objetivo: registrar atividade e encaminhar comprovante para avaliação. Gatilho: selecionar Nova atividade.

• 1. O sistema apresenta formulário e seletor de arquivo.

• 2. O aluno informa nome, instituição, categoria, período e horas; seleciona comprovante.

• 3. O frontend apresenta resumo para revisão e confirmação.

• 4. A API valida dados e conteúdo, armazena o arquivo e calcula SHA-256.

• 5. O sistema bloqueia certificado idêntico já usado em outra atividade do mesmo aluno.

• 6. Na confirmação, o sistema preserva dados/documento do envio, gera protocolo e executa a análise.

• 7. A solicitação fica aprovada automaticamente ou pendente de revisão; o evento fica no histórico.

Alternativas: campos inválidos impedem gravação; falha no envio permite tentar novamente; arquivo inválido exige substituição. Reenvio repetido sem correção aberta é bloqueado. Um comprovante visualmente válido, porém ilegível para extração, pode seguir para revisão manual.

Pós-condições: a atividade e a solicitação estão vinculadas ao aluno, com protocolo e evidências. Envio não significa aprovação.

4.1 UC02 — Realizar análise automática

Ator: sistema. Pré-condições: atividade própria e comprovante armazenado com integridade confirmada.

Fluxo: extrair texto e campos; selecionar regra vigente; comparar nome, atividade, instituição, período, horas e restrições da regra; registrar confiança, explicação e resultado; aprovar ou encaminhar para revisão; registrar status no histórico.

Alternativas: falta de regra, divergência, dados insuficientes e indisponibilidade do OCR levam à revisão manual. A extração lê até três páginas do PDF. A confiança é a fração de verificações coincidentes; não certifica autenticidade nem representa probabilidade estatística de fraude.

5. UC03 — Revisar solicitação manualmente

Ator: avaliador autorizado da instituição do aluno. Pré-condição: pedido aguardando revisão. Gatilho: abrir item da fila.

• 1. O sistema exibe dados, certificado associado, extração, comparações e regra utilizada.

• 2. O avaliador confere o material e escolhe aprovar, reprovar ou solicitar correção.

• 3. Informa justificativa e confirma a ação no diálogo da aplicação.

• 4. A API bloqueia o pedido na transação, salva decisão/responsável e muda o status.

• 5. O histórico recebe um evento e o aluno passa a consultar a justificativa.

Alternativas: pedido já decidido retorna conflito; acesso de outro perfil/instituição é bloqueado; arquivo inexistente ou com integridade divergente não é entregue. Erros de gravação mantêm a justificativa na interface para nova tentativa. Não há distribuição prévia de pedidos entre avaliadores; a exclusão mútua ocorre ao decidir.

Pós-condições: decisão persistida e auditável. Se houver correção, o aluno pode editar os dados e anexar comprovante para novo envio; os dados anteriores permanecem registrados na submissão original.

5.1 UC04 — Consultar status e histórico

Ator: aluno autenticado. Fluxo: abrir Minhas solicitações; pesquisar atividade/protocolo e filtrar status; navegar por páginas de dez itens; abrir detalhe; visualizar protocolo, dados, resultado, justificativa, comprovante e linha do tempo; quando houver correção, acessar Corrigir e reenviar.

Alternativas: lista vazia orienta novo cadastro; busca sem correspondência exibe estado vazio; acesso a pedido alheio é bloqueado; arquivo indisponível mantém os demais dados da consulta. Pós-condição: consulta simples não altera o pedido. Os rótulos do aluno agrupam processamento e revisão em Em análise; os demais são Aprovada, Reprovada, Correções e Rascunho.

6. UC05 — Gerenciar regras de validação

Ator: administrador da instituição. Objetivo: manter critérios de avaliação com histórico de versões. Gatilho: abrir a área administrativa.

• 1. O sistema lista a versão mais recente de cada categoria.

• 2. O administrador cadastra categoria nova ou edita uma regra existente.

• 3. Define vigência, limite de horas, documentos aceitos, limite de confiança e permissão de aprovação automática.

• 4. A API verifica preenchimento, datas e conflitos de categoria.

• 5. A gravação cria versão 1 para categoria nova, ou a próxima versão para uma edição.

• 6. As solicitações já processadas continuam ligadas ao registro anterior; novas análises consultam as versões aplicáveis.

Alternativas: categoria duplicada é rejeitada; edição de versão desatualizada retorna conflito; mudar a categoria exige novo cadastro. Histórico de versões disponível pela API GET /api/v1/rules/{id}/versions. A interface lista a versão atual, sem exclusão física de critérios utilizados.

6.1 Modelo e persistência da Iteração 2

Entidades principais: Institution, User, Course, ComplementaryActivity, CertificateDocument, Submission, ValidationRule, AutomaticAnalysis, ManualReview e StatusHistory. Uma atividade possui documentos e múltiplas submissões; cada submissão referencia seu comprovante, guarda activity_snapshot e pode referenciar uma versão da regra. Cada submissão contém análise automática, revisões e eventos de status.

A migração 20261104_0002 acrescenta document_id e activity_snapshot. O preenchimento de registros existentes utiliza o estado disponível no banco; não reconstrói dados já sobrescritos antes da migração. Os diagramas de domínio, classes e sequências anteriores estão preservados em imgs e nos documentos anteriores; os novos vínculos descritos nesta seção complementam essa modelagem.

7. Arquitetura, execução e rastreabilidade

Backend: FastAPI, Pydantic, SQLAlchemy e Alembic. Banco: PostgreSQL. Frontend: React com Vite. Extração: PyMuPDF e Tesseract. Docker Compose organiza banco, API e interface. O README contém os comandos para instalação limpa, contas de demonstração, testes e encerramento.

Fluxo de submissão: React → controller de atividades → SubmissionService → OCR/AnalysisService → repositories → PostgreSQL. Fluxo de revisão: React → controller de revisão → ReviewService → bloqueio transacional → ManualReview/StatusHistory. Fluxo de regra: controller → RuleService → nova versão imutável.

Requisito / caso	Componentes e evidência
RF01 / acesso	controllers/auth.py; api/deps.py; Login e ProtectedRoute
RF02–05 / UC01	controllers/activities.py; document_service.py; NewActivity
RF06–09 / UC02	ocr_service.py; analysis_service.py; submission_service.py
RF10 / UC03	review_service.py; review_repository.py; ReviewDetail
RF11 / UC04	ActivityDetail; Dashboard; RequestDetail
RF12 / UC05	rule_service.py; rule_repository.py; AdminDashboard
RF13 / histórico	Submission; ManualReview; StatusHistory; testes de regressão

7.1 Referências de controle de versão

v1 preserva a Iteração 1. iteracao2 mantém o exercício de duas alterações, integrado à master pelo PR nº 30. v2 identifica o código e a documentação desta entrega. A autoria e o histórico originais do Grupo 05 foram preservados no repositório principal.

Repositório: https://github.com/murilloohs/ppads-grupo-05
Tag da entrega: https://github.com/murilloohs/ppads-grupo-05/tree/v2
Merge do exercício: https://github.com/murilloohs/ppads-grupo-05/pull/30

8. Telas da implementação

As figuras mostram a aplicação local com contas e dados sintéticos de demonstração. A identidade visual existente foi mantida. As capturas substituem a indicação anterior de que as telas não haviam sido fornecidas.

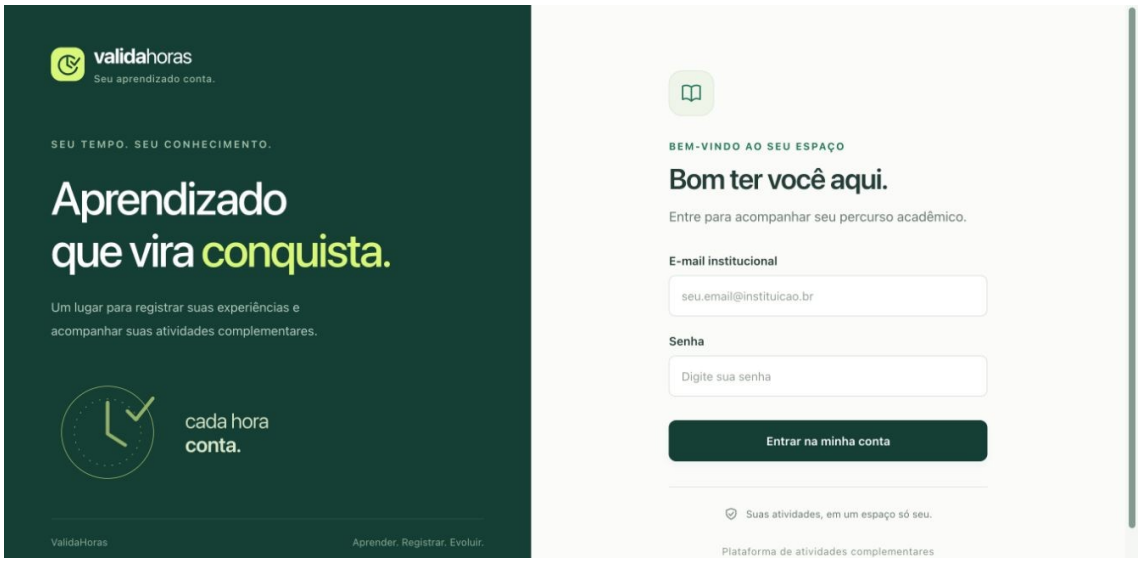


Figura 1 — Autenticação dos três perfis.

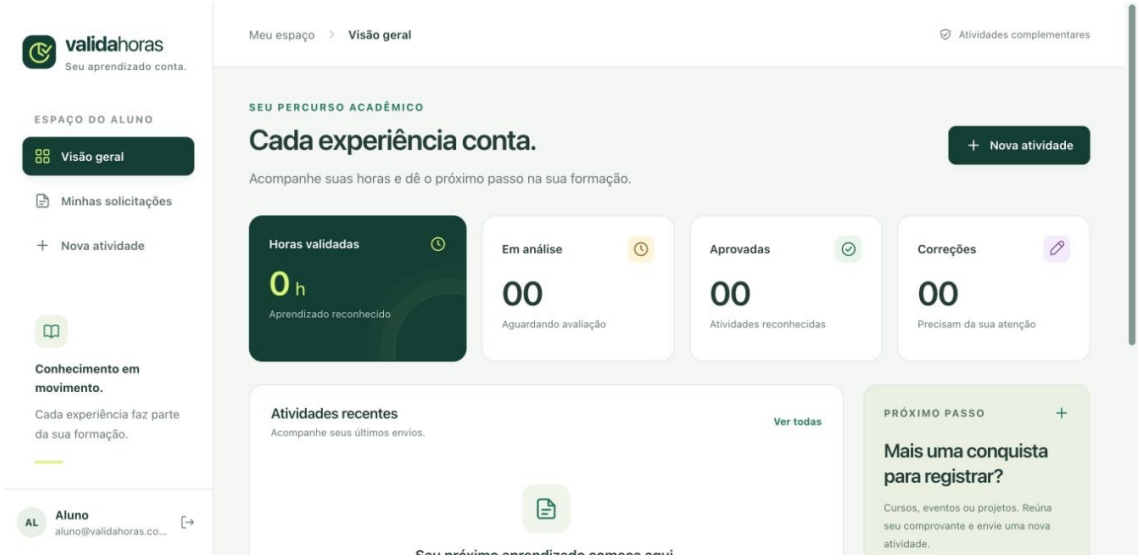


Figura 2 — Área do aluno e resumo de solicitações.

9. Detalhe, parecer e acompanhamento

A consulta apresenta os dados da atividade, protocolo, resultado, comprovante e histórico. O exemplo foi submetido pela API local com PDF digital válido e recebeu aprovação automática por coincidência com a regra cadastrada.

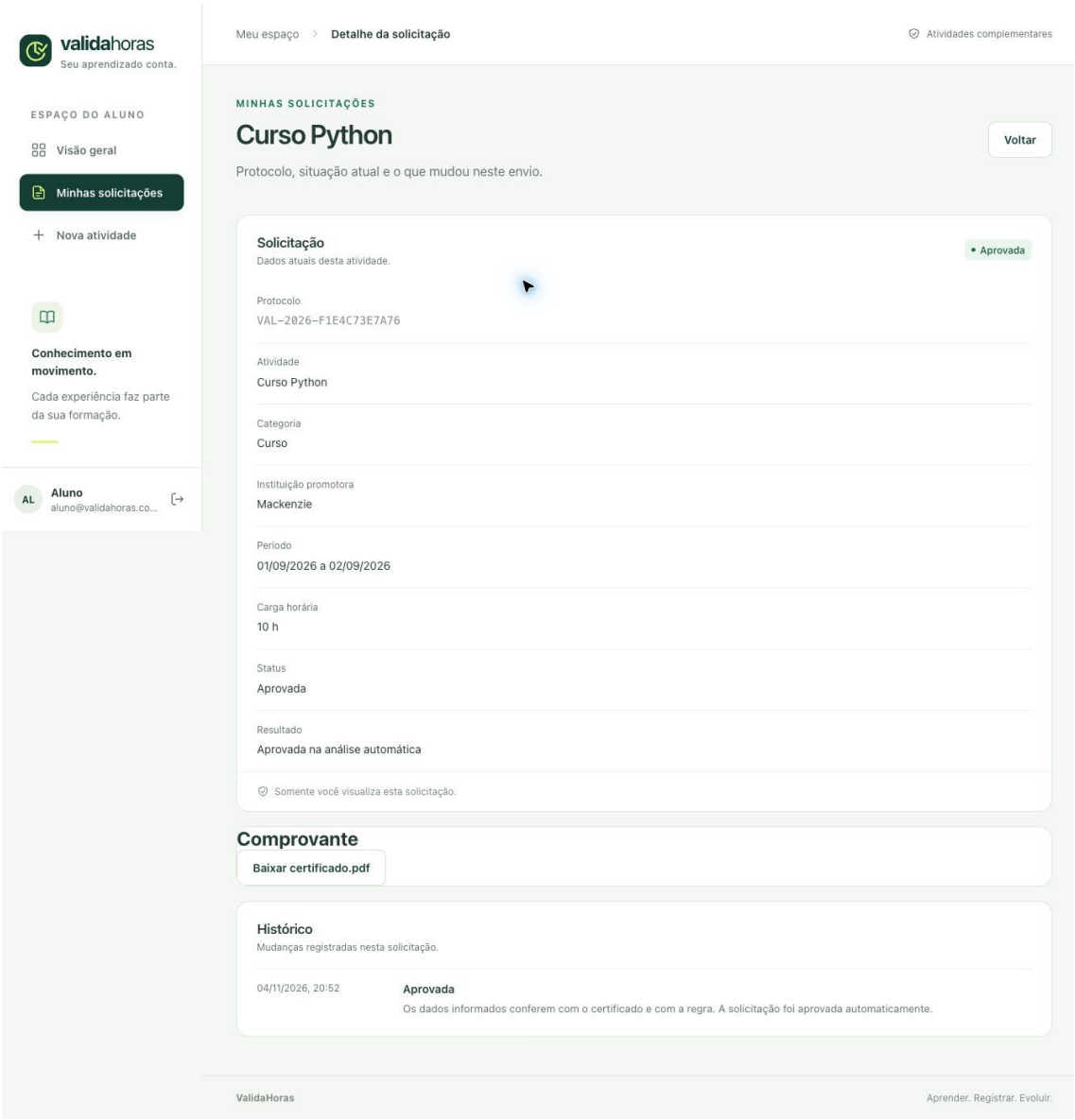


Figura 3 — Solicitação sintética aprovada e histórico disponível ao aluno.

9.1 Navegação dos demais perfis

O administrador acessa /admin para cadastrar critérios e criar versões. O avaliador acessa /avaliador, abre /avaliador/solicitacoes/{id}, consulta o comprovante e registra decisão. O aluno recebe orientação de correção no detalhe e usa /nova-atividade?corrigir={id} para novo envio.

10. Verificação e conclusão da entrega

Verificações executadas: migrações Alembic em banco PostgreSQL isolado; 15 testes automatizados aprovados; Ruff aprovado; ESLint aprovado; build de produção do frontend aprovado; auditoria estática do contrato de UI sem achados; análise npm sem vulnerabilidades após atualização compatível das dependências.

Foram cobertos autenticação e perfis, acesso próprio, upload, extração de campos, análise automática, fila e decisão, correção/reenvio, preservação do comprovante e dos dados anteriores, imutabilidade de regras, edição desatualizada, duplicidade e alteração indevida de arquivo. A submissão de um PDF digital válido também foi exercitada contra a API em execução e consultada no navegador.

Limites de verificação: os testes locais usaram Python 3.12 e PostgreSQL isolado, enquanto o Dockerfile define Python 3.13 e o Compose PostgreSQL 17. Não foi executado o build das imagens Docker nesta máquina, que não possui Docker instalado. O OCR de imagem tem cobertura simulada e Tesseract é instalado na imagem; não foi executada leitura real de imagem por Tesseract local. O envio de arquivo pelo seletor do navegador não foi concluído na automação, mas o upload real e o fluxo de envio foram testados pela API e a interface de consulta foi inspecionada.

O produto entregue inclui os fluxos centrais do ValidaHoras, documento revisado, instruções reproduzíveis e identificação de versão. As metas de produção e integrações futuras permanecem separadas do escopo acadêmico validado. O arquivo docs/VALIDACAO.md registra os comandos e resultados; o README orienta a reprodução e o documento COMO_ENTREGAR.md separa os itens de cada atividade.